Trust-But-Verify: Poisoning-Resilient Locally Private Graph Learning Protocols

Longzhu He[§], Li Sun[§,*], Hao Peng[†], Ruijie Wang[†], Raymond Chi-Wing Wong[‖], Sen Su[§,‡,*]
[§]Beijing University of Posts and Telecommunications [†]Beihang University
[‖]The Hong Kong University of Science and Technology [‡]Chongqing University of Posts and Telecommunications
{helongzhu,lsun,susen}@bupt.edu.cn; {penghao,ruijiew}@buaa.edu.cn; raywong@cse.ust.hk

***Abstract*—Built upon local differential privacy (LDP), locally private graph learning protocols have emerged as an important paradigm for decentralized graph learning, balancing privacy protection and learning utility. Under such protocols, each user locally perturbs their node features and adjacency information before transmission, ensuring formal privacy guarantees without original data leaving the device. However, the inherently open participation nature renders these protocols critically vulnerable to data poisoning attacks, where adversaries inject carefully crafted malicious nodes to corrupt neighborhood aggregation and degrade downstream utility. Despite the severity of this threat, effective defenses in this setting remain largely unexplored. In this paper, we propose VERITAS, a poisoning-resilient locally private graph learning protocol built on a *trust-but-verify* paradigm. By introducing a verification list encoding graded peer trust levels, VERITAS jointly privatizes node features and graph structure on the user side, while exploiting bilateral attestation asymmetry on the server side to identify and prune malicious nodes. Concretely, VERITAS comprises four synergistic stages: ① *local data perturbation*, ② *attestation-driven malicious node pruning*, ③ *utility restoration via dual denoising*, and ④ *robust private graph learning*. Extensive experiments on four real-world benchmark datasets across multiple LDP mechanisms and GNN architectures demonstrate that VERITAS effectively defends against data poisoning attacks and significantly improves downstream graph learning utility under rigorous privacy guarantees.**

***Index Terms*—local differential privacy, graph learning, data poisoning attack, privacy preservation, robustness**

I. INTRODUCTION

Graph neural networks (GNNs) [1]–[3] have emerged as a cornerstone tool in data mining, exhibiting strong representation learning capabilities across a wide range of graph mining tasks, including node classification [2], link prediction [4], and graph clustering [5]. Their success has also enabled numerous real-world applications, such as recommendation systems [6], social network analysis [7], and fraud detection [8]. However, the graph-structured data underlying these applications is often highly sensitive, with node features containing private user information and edges revealing confidential social relationships. Previous studies [9]–[12] have demonstrated that GNNs are vulnerable to privacy inference attacks, which can expose sensitive information from both node features and graph topology. Such threats have raised growing concerns regarding data confidentiality and highlighted the urgent need for privacy-preserving graph learning techniques that effectively protect individual privacy while maintaining strong learning utility.

[*]Correspond to: Sen Su and Li Sun.

To address the growing privacy concerns in graph learning, *locally private graph learning protocols* [13]–[23], built upon local differential privacy (LDP) [24], [25], have attracted considerable attention from the security and privacy communities. Unlike centralized privacy-preserving approaches that require users to trust a data curator, LDP adopts a stronger trust model in which each user perturbs their sensitive data locally before any transmission, thereby providing rigorous privacy guarantees without relying on any trusted third party. Thanks to these appealing properties, LDP has been widely adopted by major technology companies, including Google [26], Apple [27], and Microsoft [28]. As illustrated in Fig. 1(a), each user independently perturbs their sensitive data before sending noisy reports to an untrusted server, which subsequently performs private graph learning on the collected noisy data. By ensuring that original user data never leaves local devices, this decentralized learning paradigm offers strong user-level privacy protection while retaining the utility of downstream graph learning tasks.

However, the open-participation nature of locally private graph learning renders it particularly vulnerable to *data poisoning attacks* [16], as illustrated in Fig. 1(b). In realistic distributed scenarios, an adversary can create sybil accounts [29], establish plausible social identities [30], and infiltrate the graph by forming links with legitimate users [31]. Once admitted, these malicious nodes submit carefully crafted perturbed feature and topology reports to the server, thereby injecting adversarial signals into the training data. Through iterative message passing, such poisoned information propagates through the graph and contaminates the learned node representations, ultimately degrading the performance of downstream tasks, such as node classification. Critically, local perturbation obscures the distinction between benign and malicious reports, making it extremely challenging for the server to identify adversarial participants based solely on the received data.

Despite the severity of this threat, effective defenses remain largely unexplored. Designing such defenses is highly non-trivial for three reasons. *First*, the decentralized nature of LDP prevents the server from accessing raw features or adjacency information, rendering conventional anomaly detection and robust aggregation methods inapplicable. *Second*, the noise introduced by LDP mechanisms obscures malicious manipulations, blurring the boundary between benign perturbation and adversarial injection, making it difficult to distinguish malicious nodes from honest ones based on noisy observations

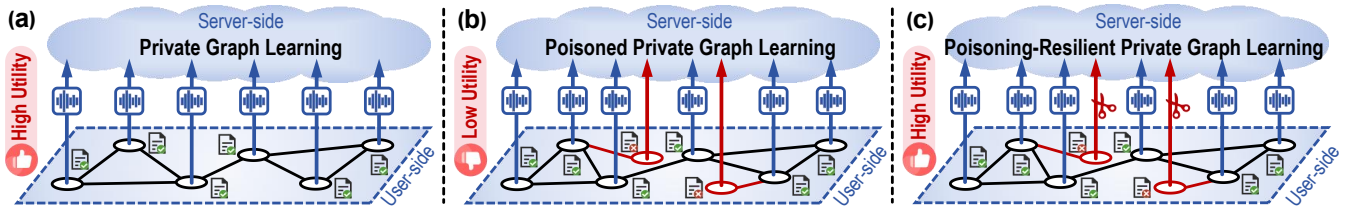


Fig. 1. **Motivation and core intuition of VERITAS. (a)** Locally private graph learning protocol. Multiple users independently perturb their sensitive node features and adjacency information via local differential privacy (LDP) mechanisms. The resulting noisy reports are transmitted to an untrusted server, which performs graph learning on the collected perturbed data. **(b)** Data poisoning attacks against locally private graph learning, where red nodes denote injected malicious participants that submit carefully crafted poisoned reports to corrupt the training process and degrade downstream task performance. **(c)** VERITAS detects and prunes malicious nodes through bilateral attestation, preserving graph learning utility while maintaining rigorous local privacy guarantees.

alone. *Third*, existing graph poisoning defenses [30], [32], [33] are developed for centralized graph learning and assume access to unperturbed graph data, assumptions that are fundamentally violated under local privacy protection.

To address the above challenges, we develop a poisoning-resilient locally private graph learning protocol, termed VERITAS[1], as shown in Fig. 1(c). Following a *trust-but-verify* paradigm, VERITAS initially trusts all user-submitted reports while subsequently verifying their credibility through a verification list that encodes graded peer trust levels. This design enables the server to exploit the inherent asymmetry in bilateral attestation signals to identify and prune malicious nodes while preserving rigorous LDP guarantees on the user side. Concretely, our VERITAS comprises four synergistic stages: ① *local data perturbation* for privacy-preserving data collection, ② *attestation-driven malicious node pruning* for verifying structural consistency and eliminating adversarial influence, ③ *utility restoration via dual denoising* to mitigate perturbation-induced noise, and ④ *robust private graph learning* for downstream graph learning. By seamlessly coupling these components, VERITAS establishes an end-to-end defense pipeline that effectively counters data poisoning attacks while maintaining rigorous privacy guarantees. The main contributions of this paper are summarized as follows:

- ◇ **Important problem.** We investigate the problem of poisoning attacks against locally private graph learning, highlighting a critical challenge and motivating the development of secure, robust, and privacy-preserving graph learning.
- ◇ **Novel defense framework.** We develop VERITAS, a poisoning-resilient locally private graph learning protocol built upon a trust-but-verify paradigm. VERITAS introduces a four-stage defense pipeline that jointly enhances poisoning resilience, privacy preservation, and graph learning utility.
- ◇ **Extensive experiments.** Experiments on real-world datasets across multiple LDP mechanisms and GNN architectures demonstrate that VERITAS effectively defends against data poisoning attacks and significantly improves learning utility.

II. PRELIMINARIES

This section first defines the problem (▷ Sect. II-A), then provides the essential background on local differential privacy (▷ Sect. II-B) and locally private graph learning (▷ Sect. II-C).

[1] *Veritas* is the Latin word for "truth".

A. Problem Definition

Consider a graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ denotes the set of nodes. The feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ associates each node $v \in \mathcal{V}$ with a d-dimensional feature vector $\mathbf{x}_v \in \mathbb{R}^d$, and the adjacency matrix $\mathbf{A} \in \{0,1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ encodes the graph structure, where each row $\mathbf{a}_v \in \{0,1\}^{|\mathcal{V}|}$ is the adjacency vector of node v. In this work, we consider a comprehensive local privacy setting in which both node features and structural information are treated as sensitive. Each user v locally holds the private pair $(\mathbf{x}_v, \mathbf{a}_v)$, capturing its attribute and relational information, respectively. A central server coordinates the learning process but is not assumed to be fully trustworthy; hence, neither $\mathbf{X}$ nor $\mathbf{A}$ is directly exposed to it. Instead, each user perturbs its local data via LDP mechanisms [13] prior to transmission, and the server performs graph learning on the collected noisy reports—forming a two-phase pipeline of *user-side perturbation* and *server-side learning*. However, the decentralized nature of locally private graph learning renders it inherently vulnerable to data poisoning attacks [16], which can severely degrade the utility of downstream tasks (see Sect. III for the threat model). Our goal is therefore to develop a poisoning-resilient locally private graph learning framework that achieves accurate downstream graph learning while providing rigorous privacy guarantees.

B. Local Differential Privacy

Local differential privacy (LDP) [24], [34] has been extensively studied and widely deployed in decentralized data collection and analysis scenarios. In practice, it has been adopted by major industry systems, including Apple [27], Google [26], and Microsoft [28]. In the LDP setting, a server interacts with multiple users, each holding sensitive data. Rather than sharing raw data with an untrusted server, each user locally perturbs its data via a randomized mechanism $\mathcal{M}$ before transmission. The server then performs data analysis and learning based solely on the perturbed data, thereby providing formal privacy guarantees. The formal definition of ϵ-LDP is provided below:

Definition 1 (ϵ-LDP [24]). *A local perturbation mechanism $\mathcal{M}$ satisfies ϵ-LDP, where $\epsilon > 0$, if for any two inputs x, x' and any output $y \in \mathrm{Range}(\mathcal{M})$, it holds that*

$$\Pr[\mathcal{M}(x) = y] \leq e^{\epsilon} \cdot \Pr[\mathcal{M}(x') = y], \tag{1}$$

where ϵ is the "privacy budget" that controls the privacy-utility trade-off: a smaller ϵ provides stronger privacy protection but typically leads to greater utility degradation. Range($\mathcal{M}$) denotes the output space of $\mathcal{M}$. In graph settings, protecting relational information requires extending LDP to link-level data. The resulting notion, known as ϵ-link LDP [14], [21], [35], is defined as follows:

Definition 2 (ϵ-link LDP [35]). *A randomized mechanism $\mathcal{M}$ satisfies ϵ-link LDP, if for any node v and for any two adjacency vectors $\mathbf{a}_v, \mathbf{a}'_v \in \{0,1\}^{|\mathcal{V}|}$ differing in exactly one coordinate (i.e., $\|\mathbf{a}_v - \mathbf{a}'_v\|_1 = 1$) and for any measurable set $\mathcal{S} \subseteq \mathrm{Range}(\mathcal{M})$, it holds that*

$$\Pr\big[\mathcal{M}(\mathbf{a}_v) \in \mathcal{S}\big] \leq e^{\epsilon} \cdot \Pr\big[\mathcal{M}(\mathbf{a}'_v) \in \mathcal{S}\big]. \tag{2}$$

Intuitively, this definition ensures that an observer cannot reliably infer the presence or absence of any individual edge, thereby providing formal protection for relational information. Beyond these definitions, LDP satisfies two fundamental properties that are instrumental in our framework, as follows:

Theorem 1 (Post-Processing [25]). *If $\mathcal{M} : \mathcal{X} \to \mathcal{Z}$ satisfies ϵ-LDP, then for any (possibly randomized) function $\mathcal{F} : \mathcal{Z} \to \mathcal{Z}'$, the composition $\mathcal{F} \circ \mathcal{M}$ also satisfies ϵ-LDP.*

Theorem 2 (Sequential Composition [25]). *Let $\mathcal{M}_i : \mathcal{X} \to \mathcal{Z}_i$ satisfy ϵ_i-LDP for $i \in \{1, 2, \ldots, n\}$. Applying all mechanisms to the same data yields a joint mechanism $\mathcal{M} = (\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_n)$ that satisfies $(\sum_{i=1}^{n} \epsilon_i)$-LDP.*

C. Locally Private Graph Learning

We now formalize the locally private graph learning pipeline $\mathbb{P}_{\mathcal{M},\mathcal{F}_\theta}$, where $\mathcal{M}$ denotes the user-side perturbation mechanism and $\mathcal{F}_\theta$ represents the server-side graph learning model parameterized by θ, detailed as follows:

1) User-Side Perturbation: Each node v locally perturbs its private data before transmitting to the server. Specifically, the node feature vector $\mathbf{x}_v \in \mathbb{R}^d$ and the binary adjacency vector $\mathbf{a}_v \in \{0,1\}^{|\mathcal{V}|}$ are processed via two perturbation mechanisms:

$$\mathbf{x}'_v \leftarrow \mathcal{M}_x(\mathbf{x}_v, \epsilon_x), \qquad \mathbf{a}'_v \leftarrow \mathcal{M}_a(\mathbf{a}_v, \epsilon_a), \tag{3}$$

where ϵ_x and ϵ_a denote the privacy budgets for node feature and link perturbation, respectively, satisfying $\epsilon_x + \epsilon_a = \epsilon$. We next detail the instantiation of these two mechanisms.

◇ *Node Feature Perturbation.* For a d-dimensional node feature vector $\mathbf{x}_v$, the perturbation proceeds in two steps: ① a subset $\mathcal{S} \subseteq \{1, 2, \ldots, d\}$ of size m is sampled uniformly; ② each selected dimension $i \in \mathcal{S}$ is independently perturbed via an (ϵ_x/m)-LDP mechanism, while the remaining $d-m$ dimensions are set to zero. Beyond classical LDP mechanisms such as the Laplace mechanism [36], the Gaussian mechanism [37], and the one-bit mechanism [28], representative LDP mechanisms for node feature include the multi-bit mechanism [13], the square wave mechanism [38], and the piecewise mechanism [39]. As an illustrative example, we describe the one-dimensional piecewise mechanism (PM). Given an input $x \in [\alpha, \beta]$, the perturbed output x' is sampled from the bounded interval $[-\mathcal{B}, \mathcal{B}]$, where $\mathcal{B} = \frac{e^{\epsilon_x/2}+1}{e^{\epsilon_x/2}-1}$, according to the following density:

$$\Pr[x' = c \mid x] = \begin{cases} p, & \text{if } c \in [l(x),\, r(x)], \\ p/e^{\epsilon_x}, & \text{if } c \in [-\mathcal{B},\, l(x)) \cup (r(x),\, \mathcal{B}], \end{cases} \tag{4}$$

where

$$p = \frac{e^{\epsilon_x} - e^{\epsilon_x/2}}{2e^{\epsilon_x/2}+2}, l(x) = \frac{\mathcal{B}+1}{2}x - \frac{\mathcal{B}-1}{2}, r(x) = l(x) + \mathcal{B} - 1. \tag{5}$$

◇ *Adjacency List Perturbation.* To protect the binary adjacency vector $\mathbf{a}_v$, we employ the randomized response (RR) mechanism [40]. Each entry $a_{v,u} \in \{0,1\}$ is independently flipped with probability inversely proportional to e^{ϵ_a}:

$$a'_{v,u} = \begin{cases} a_{v,u}, & \text{with probability } e^{\epsilon_a}/(e^{\epsilon_a}+1), \\ 1 - a_{v,u}, & \text{with probability } 1/(e^{\epsilon_a}+1). \end{cases} \tag{6}$$

By Theorem 2, the combined mechanism $(\mathcal{M}_x, \mathcal{M}_a)$ satisfies ϵ-LDP with $\epsilon = \epsilon_x + \epsilon_a$. Consequently, the server only has access to samples drawn from a perturbed data distribution induced by $\mathcal{M}$, rather than the true underlying data. The resulting perturbed data is then transmitted to the server, with no raw private data ever leaving the user's local device.

2) Server-Side Learning: After receiving the perturbed reports $\{(\mathbf{x}'_v, \mathbf{a}'_v)\}_{v \in \mathcal{V}}$, the server reconstructs a noisy graph $\mathcal{G}' = (\mathcal{V}, \mathbf{A}', \mathbf{X}')$, where $\mathbf{X}' = [\mathbf{x}'_1, \mathbf{x}'_2, \ldots, \mathbf{x}'_{|\mathcal{V}|}]^\top$ and $\mathbf{A}'$ is induced from $\{\mathbf{a}'_v\}_{v \in \mathcal{V}}$. Based on $\mathcal{G}'$, the server trains a K-layer graph neural network (GNN) [1], [2] for downstream tasks such as node classification. Due to local perturbation, the true neighborhood $\mathcal{N}(v)$ is not directly observable; instead, the server operates on a perturbed neighborhood defined as $\widehat{\mathcal{N}}(v) = \{u \mid a'_{v,u} = 1\}$. The node representations are then learned via iterative neighborhood aggregation. Specifically, at the k-th layer, the embedding of node v is computed as:

$$\mathbf{h}^k_{\widehat{\mathcal{N}}(v)} = \textsc{Aggregate}_k(\{\mathbf{h}^{k-1}_u \mid u \in \widehat{\mathcal{N}}(v)\}), \tag{7}$$

$$\mathbf{h}^k_v = \textsc{Update}_k(\mathbf{h}^k_{\widehat{\mathcal{N}}(v)}), \tag{8}$$

where $\textsc{Aggregate}_k(\cdot)$ is a permutation-invariant function (e.g., mean or sum), and $\textsc{Update}_k(\cdot)$ is a learnable non-linear transformation. The input layer is initialized as $\mathbf{h}^0_v = \mathbf{x}'_v$. After K layers, the learned representations $\mathbf{h}^K_v$ encode multi-hop structural information from the perturbed graph. The model is trained end-to-end on labeled nodes $\mathcal{V}_l$ and subsequently used to perform inference on unlabeled nodes $\mathcal{V}_u = \mathcal{V} \setminus \mathcal{V}_l$.

III. Threat Model

In the locally differentially private graph learning setting, we consider two categories of security threats that may coexist: ① *privacy inference attacks* conducted by a semi-honest server (▷ Sect. III-A), and ② *data poisoning attacks* launched by a malicious external attacker (▷ Sect. III-B).

A. Privacy Inference Attacks by Semi-Honest Server

Under the semi-honest adversary model, the server faithfully adheres to the prescribed protocol $\mathbb{P}$ yet may simultaneously attempt to infer individual users' private information from their perturbed reports. Despite the rigorous probabilistic guarantees of LDP, a curious server can still exploit interactions with the trained GNN models to launch *attribute inference attacks* [9], [11], *link inference attacks* [9], [10], or *membership inference attacks* [12]. Such threats underscore the necessity of strict ϵ-LDP requirements imposed on both features and links: even if the server aggregates all received data, the privacy of any individual user's raw input remains formally protected.

B. Data Poisoning Attacks by Malicious Attacker

Beyond inference-based privacy threats, the locally private graph learning protocol $\mathbb{P}$ is also vulnerable to data poisoning attacks [16], as shown in Fig. 1(b) and Alg. 1. In such attacks, the attacker injects up to $n_{\max}$ malicious nodes into the original graph $\mathcal{G}$ under a predefined attack budget Δ, and strategically constructs adversarial feature and adjacency reports according to a set of attack strategies $\{\mathbb{S}_x, \mathbb{S}_{\text{target}}, \mathbb{S}_{\text{atk}}\}$. The objective is to compromise the utility of downstream graph learning tasks. Despite this significant threat, effective defenses against such poisoning attacks remain largely underexplored in prior work. We formally characterize the attacker along three dimensions:

1) Adversary Capability: The attacker is capable of injecting up to $n_{\max}$ fake nodes, denoted by $\mathcal{V}_{\text{atk}}$ (with $1 < |\mathcal{V}_{\text{atk}}| \ll |\mathcal{V}|$), into the original graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, and connecting them to a set of target victim nodes $\mathcal{V}_t \subseteq \mathcal{V}$. Each fake node $v_{\text{atk}} \in \mathcal{V}_{\text{atk}}$ is fully controlled by the attacker, who can arbitrarily construct its node feature vector $\tilde{\mathbf{x}}^\dagger_{v_{\text{atk}}}$ and adjacency vector $\tilde{\mathbf{a}}^\dagger_{v_{\text{atk}}}$ according to a set of attack strategies $\{\mathbb{S}_x, \mathbb{S}_{\text{target}}, \mathbb{S}_{\text{atk}}\}$. Specifically, $\mathbb{S}_x$ governs the generation of malicious features to manipulate the learned representations, $\mathbb{S}_{\text{target}}$ determines how fake nodes establish connections with target victim nodes to directly influence their local neighborhoods, and $\mathbb{S}_{\text{atk}}$ further optimizes the connectivity among fake nodes to amplify the overall attack effect. Importantly, since fake nodes masquerade as legitimate participants, the server cannot reliably distinguish them from genuine users based solely on the received (already perturbed) reports. By establishing links with victim nodes, the injected fake nodes effectively introduce adversarial neighbors into their aggregation neighborhoods, thereby influencing the downstream message-passing process.

2) Adversary Knowledge: We assume a realistic *black-box* threat model: the attacker is assumed to have access only to the publicly available protocol $\mathbb{P}$, including the perturbation mechanisms $\mathcal{M}_x$ and $\mathcal{M}_a$, as well as the associated privacy budgets ϵ_x and ϵ_a, since these parameters are typically transparent to all participants. Crucially, the attacker has no access to the raw node features, ground-truth labels, or the exact neighborhood information of genuine users. Furthermore, the internal states of the server-side GNN, such as the model architecture, parameters, and training dynamics, remain completely inaccessible to the attacker. This assumption is consistent with previous studies [16] on data poisoning attacks in the locally private graph learning protocol $\mathbb{P}$, and reflects a realistic scenario in which the attacker must create malicious inputs without access to structural knowledge or feedback from the target model.

Algorithm 1 Data Poisoning Attack Against Protocol $\mathbb{P}$

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, target node set $\mathcal{V}_t$, parameter $n_{\max}$, protocol $\mathbb{P}$, attacker strategies $\{\mathbb{S}_x, \mathbb{S}_{\text{target}}, \mathbb{S}_{\text{atk}}\}$.
Output: Fake node set $\mathcal{V}_{\text{atk}}$, reports $\{(\tilde{\mathbf{x}}^\dagger_v, \tilde{\mathbf{a}}^\dagger_v)\}_{v \in \mathcal{V}_{\text{atk}}}$.

```
Initialize V_atk ← ∅.                          ▷ Create a malicious node set
for i = 1 to n_max do
    Create a fake node v_atk.
    x̃†_{v_atk} ← S_x(P).                       ▷ Craft malicious node feature vector
    if V_t ≠ ∅ then
        Select a subset of target nodes T_i ⊆ V_t.
        V_t ← V_t \ T_i.                       ▷ Avoid repeated targeting
    end if
    ã†_{v_atk} ← S_target(T_i, V).             ▷ Construct adjacency links
    V_atk ← V_atk ∪ {v_atk}.
end for
{ã†_v}_{v∈V_atk} ← S_atk(V_atk).               ▷ Fake-fake coordination
return {(x̃†_v, ã†_v)}_{v∈V_atk}.
```

3) Adversary Objective: The attacker aims to degrade the learning utility of the GNN model $\mathcal{F}_\theta$ trained on the poisoned graph. Let $\mathcal{G}^\dagger = (\mathcal{V} \cup \mathcal{V}_{\text{atk}}, \mathbf{A}^\dagger, \mathbf{X}^\dagger)$ denote the poisoned graph after node injection. The attacker's objective is to optimize the features and links of the injected nodes to maximize an impact metric $\mho(\cdot)$, while remaining within a stealth attack budget Δ to avoid detection. Formally, this objective is defined as:

$$\max_{\mathcal{V}_{\text{atk}}, \mathbf{X}^\dagger, \mathbf{A}^\dagger} \mho(\mathcal{F}_\theta(\mathcal{G}^\dagger)) \;\; \textit{s.t.}\; \text{dist}\big((\mathbf{X}, \mathbf{A}), (\mathbf{X}^\dagger, \mathbf{A}^\dagger)\big) \leq \Delta, \quad (9)$$

where the $\text{dist}(\cdot, \cdot)$ function quantifies the magnitude of perturbations to enforce the stealthiness of the attack. In the context of node classification [2], the metric $\mho(\cdot)$ usually corresponds to increasing the misclassification rate.

IV. Methodology

In this section, we introduce VERITAS, a poisoning-resilient protocol designed for locally private graph learning. Built upon a *trust-but-verify* paradigm, VERITAS allows all users to participate in the learning process by first *trusting* their local reports while subsequently *verifying* their consistency and credibility on the server side. As illustrated in Fig. 2 and Alg. 3, VERITAS integrates four synergistic stages: ① *local data perturbation* for private data collection, ② *attestation-driven malicious node pruning* for verifying structural consistency and eliminating adversarial influence, ③ *utility restoration via dual denoising* to mitigate perturbation-induced noise, and ④ *robust private graph learning* for downstream tasks. By seamlessly coupling these components, VERITAS establishes an end-to-end defense pipeline that embodies the principle of "*trust first, verify later*", effectively safeguarding locally private graph learning against poisoning attacks while maintaining rigorous formal privacy guarantees. The subsequent sections (▷ IV-A-IV-D) detail each stage, with a theoretical analysis provided in Sect. IV-E.

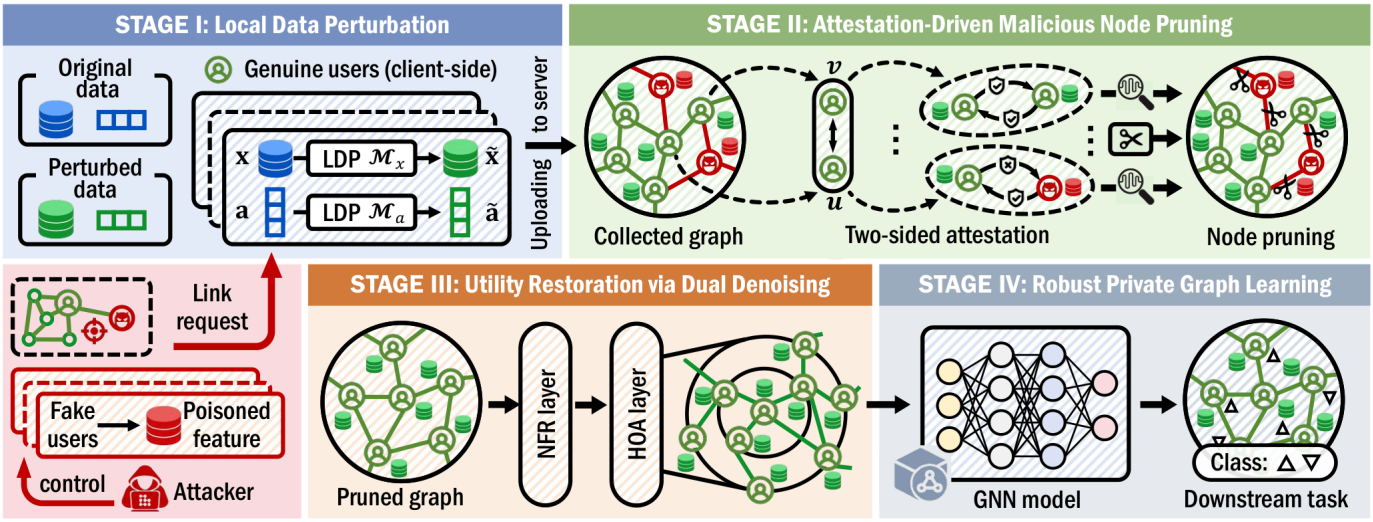


Fig. 2. **Overview of VERITAS.** VERITAS comprises four synergistic stages: ① local data perturbation for private data collection, ② attestation-driven malicious node pruning for adversarial participant removal, ③ utility restoration via dual denoising, and ④ robust private graph learning for downstream inference.

A. Local Data Perturbation

This stage performs LDP perturbation on both node features and graph structure before any data transmission. Unlike previous work [15], [16], [18] focusing solely on feature protection, we jointly privatize node features and graph structure, ensuring comprehensive privacy guarantees for graph-structured data.

1) Node Feature Perturbation: Given a d-dimensional node feature vector $\mathbf{x}_v \in [\alpha, \beta]^d$, direct access to original node features is prohibited under LDP constraints. To address this, each user v perturbs $\mathbf{x}_v$ locally using an LDP mechanism $\mathcal{M}_x$ to obtain a privatized representation $\tilde{\mathbf{x}}_v$. We adopt a unified node feature LDP perturbation pipeline, which generalizes several representative mechanisms, including PM [39], MB [13], and SW [38]. The pipeline is summarized in Alg. 2. Specifically, a perturbation budget ϵ_x is allocated to feature protection, and a subset $\mathcal{S} \subseteq [d]$ of size m is uniformly sampled (*Line 1*). Each selected dimension $i \in \mathcal{S}$ is independently perturbed using an (ϵ_x/m)-LDP mechanism, while the remaining dimensions are set to zero (*Lines 2-7*). This sampling strategy effectively reduces per-dimension noise, thereby improving utility under a fixed privacy budget. Optionally, a post-processing step $\textsc{Rect}(\cdot)$ (*e.g.*, bias correction) can be applied to ensure approximately unbiased estimation, *i.e.*, $\mathbb{E}[\tilde{\mathbf{x}}_v] \approx \mathbf{x}_v$ (*Line 8*).

2) Graph Structure Perturbation: In addition to protecting node features $\mathbf{x}_v$, we also provide privacy protection for the graph structure. Instead of adopting the conventional RR-based perturbation mechanism introduced in Sect. II-C, we propose a novel data structure, termed the *verification list*, to jointly encode relational information and peer trust, thus enhancing robustness against poisoning attacks. Specifically, each node v maintains a verification list $\breve{\mathbf{a}}_v \in \{0, 1, \ldots, \kappa\}^{|\mathcal{V}|}$, where each entry $\breve{a}_{v,u}$ represents node v's assessment of its relationship with node u, defined as follows:

$$\breve{a}_{v,u} = \begin{cases} 0 & \text{if } u \notin \mathcal{N}(v) \quad \text{(non-neighbor)}, \\ 1, 2, \ldots, \kappa & \text{if } u \in \mathcal{N}(v) \quad \text{(trust level)}. \end{cases} \tag{10}$$

The level 0 serves as a dedicated indicator of structural absence and is semantically distinct from the trust levels $\{1, 2, \ldots, \kappa\}$ assigned to the confirmed neighbors. To ensure ϵ_a-link LDP (Definition 2), each entry $\breve{a}_{v,u}$ is independently perturbed using the K-ary randomized response (KRR) mechanism [41]:

$$\Pr(\tilde{a}_{v,u} = j \mid \breve{a}_{v,u} = i) = \begin{cases} \dfrac{e^{\epsilon_a}}{e^{\epsilon_a} + \kappa} & \text{if } j = i, \\ \dfrac{1}{e^{\epsilon_a} + \kappa} & \text{if } j \neq i, \end{cases} \tag{11}$$

where ϵ_a denotes the privacy budget allocated to the perturbation of the verification list. This mechanism satisfies ϵ_a-link local differential privacy, as stated in the following proposition:

Proposition 1. *The KRR mechanism defined in Eq. (11) satisfies ϵ_a-link LDP as defined in Definition 2.*

Proof. For any two inputs $i, i' \in \{0, 1, \ldots, \kappa\}$ and any output j, the privacy ratio is bounded as

$$\frac{P(\tilde{a} = j \mid \breve{a} = i)}{P(\tilde{a} = j \mid \breve{a} = i')} \leq e^{\epsilon_a}, \tag{12}$$

which follows directly from the mechanism definition and the standard analysis of KRR mechanism [41]. □

Finally, the perturbed pair $\{(\tilde{\mathbf{x}}_v, \tilde{\mathbf{a}}_v)\}_{v \in \mathcal{V}}$ is transmitted to the server. By the composition theorem of LDP (Theorem 2), since $\tilde{\mathbf{x}}_v$ and $\tilde{\mathbf{a}}_v$ pertain to disjoint attributes, the overall perturbation satisfies ϵ-LDP with $\epsilon = \epsilon_x + \epsilon_a$.

B. Attestation-Driven Malicious Node Pruning

Upon receiving $(\tilde{\mathbf{x}}_v, \tilde{\mathbf{a}}_v)_{v \in \mathcal{V}}$, the server proceeds to assess the credibility of each reported relationship through *cross-node attestation signals*. The central insight driving this stage lies in the *incentive asymmetry* inherent in graph poisoning. A malicious node v_{atk} tends to assign high trust levels to its target neighbors to maximize its adversarial impact, whereas a benign node evaluates its peers based on local observations and

Algorithm 2 Node Feature LDP Perturbation Mechanism

Input: Node feature vector $\mathbf{x}_v \in [\alpha, \beta]^d$, coefficient δ, privacy budget $\epsilon_x > 0$.
Output: Perturbed node feature $\tilde{\mathbf{x}}_v \in [-\mathcal{B}, \mathcal{B}]^d$.

$m \leftarrow \max(1, \min(d, \lfloor \delta \cdot \epsilon_x \rfloor))$. ▷ *perturbation size*
Let $\mathbf{x}'_v \leftarrow < 0, 0, \cdots, 0 >$.
Let $\mathcal{S} \subset [d]$ denote a subset of m distinct dimensions selected uniformly at random without replacement from the index set $\{1, 2, \ldots, d\}$.
for *each sampled dimension* $i \in \mathcal{S}$ **do**
  Feed $\mathbf{x}_v[i]$ and ϵ_x/m as input to one-dimensional LDP mechanisms (*e.g.*, Eq. (4)), and obtain x'_{tmp}.
  $\mathbf{x}'_v[i] \leftarrow x'_{\text{tmp}}$. ▷ *LDP perturbation*
end for
$\tilde{\mathbf{x}}_v \leftarrow \textsc{Rect}(\mathbf{x}'_v, \epsilon_x, m, d)$. ▷ *unbiased rectification*
return Perturbed node feature $\tilde{\mathbf{x}}_v \in [-\mathcal{B}, \mathcal{B}]^d$.

naturally assigns low trust to structurally anomalous neighbors. This behavioral divergence induces a systematic asymmetry in bilateral verification entries, which can be interpreted as inconsistencies in mutual attestation. Importantly, this signal persists even under LDP noise and remains identifiable on the server. VERITAS takes advantage of this property through a three-step procedure consisting of ① *bilateral signal extraction*, ② *edge confidence scoring*, and ③ *suspicion-based pruning*.

1) Bilateral Signal Extraction: For each candidate pair (u, v), the server observes two independent perturbed entries, $\tilde{a}_{v,u}$ (node v's assessment of node u) and $\tilde{a}_{u,v}$ (node u's assessment of node v), forming a bilateral attestation signal. Based on this observation, three complementary signals are extracted:

- ◇ *Signal 1 (Structural existence).* Both parties must mutually acknowledge the relationship for an edge to be considered structurally valid: $s_{uv}^{\text{exist}} = \mathbf{1}[\tilde{a}_{v,u} \neq 0] \cdot \mathbf{1}[\tilde{a}_{u,v} \neq 0]$. An edge flagged by either party as non-existent ($\tilde{a} = 0$) is structurally unconfirmed and receives zero confidence.
- ◇ *Signal 2 (Trust consistency).* Among mutually acknowledged edges, the effective trust is determined by the lower of the two reported levels: $s_{uv}^{\text{cons}} = \min(\tilde{a}_{v,u}, \tilde{a}_{u,v})$. The min operator is a deliberate design choice grounded in adversarial reasoning: a malicious node v_{atk} will strategically report the maximum trust level κ toward all neighbors to maintain influence, so the minimum operator effectively neutralizes this manipulation; the overall trust is anchored by the honest party's conservative assessment.
- ◇ *Signal 3 (Endorsement asymmetry).* The absolute discrepancy between the two levels reported quantifies the degree of bilateral inconsistency: $s_{uv}^{\text{asym}} = |\tilde{a}_{u,v} - \tilde{a}_{v,u}|$. A high asymmetry score indicates that one party's assessment deviates substantially from the other's, which is a characteristic signature of adversarial behavior.

2) Edge Confidence Scoring: The three signals are consolidated into a unified edge confidence score $c_{uv} \in [0, 1]$:

$$c_{uv} = s_{uv}^{\text{exist}} \cdot \frac{s_{uv}^{\text{cons}}}{\kappa} \cdot \exp(-\lambda \cdot s_{uv}^{\text{asym}}), \tag{13}$$

where $\lambda > 0$ is a hyperparameter controlling the penalty for asymmetry. The three multiplicative terms operate at distinct levels of granularity: the existence term acts as a hard gate that suppresses unconfirmed edges entirely; the consistency term provides a soft, graded measure of mutual trust; and the asymmetry term exponentially penalizes one-sided endorsements. Together, Eq. (13) assigns high confidence only to edges for which both parties report comparable and positive trust levels, while systematically discounting edges that exhibit the hallmarks of adversarial manipulation.

3) Suspicion Scoring and Node Pruning: The directional differences in bilateral assessments are aggregated into a node-level suspicion score ϕ_v, which captures the systematic directional bias in v's bilateral assessments:

$$\phi_v = \frac{1}{|\hat{\mathcal{N}}_v|} \sum_{u \in \hat{\mathcal{N}}_v} (\tilde{a}_{v,u} - \tilde{a}_{u,v}), \tag{14}$$

where $\hat{\mathcal{N}}_v = \{u : \tilde{a}_{v,u} > 0 \vee \tilde{a}_{u,v} > 0\}$ denotes the set of nodes claiming any relationship with v. A positive ϕ_v indicates that v consistently assigns higher trust to others than others assign to it, precisely the behavioral fingerprint of a malicious node maximizing its structural influence while being rated poorly by honest peers.

Proposition 2 (Suspicion Score Separability)**.** *Let $v_m \in \mathcal{M}$ be a fake node that reports maximum trust $\breve{a}_{v_m,u} = \kappa$ toward all neighbors, and let each honest neighbor $u \notin \mathcal{M}$ assign trust level $\breve{a}_{u,v_m} = 1$ due to anomalous local observations. Then, after $(\kappa{+}1)$-KRR perturbation with budget ϵ_a, the expected suspicion score satisfies:*

$$\mathbb{E}[\phi_{v_m}] = (p_a - q_a)(\kappa - 1) > 0, \tag{15}$$

where $p_a = e^{\epsilon_a}/(e^{\epsilon_a} + \kappa)$ and $q_a = 1/(e^{\epsilon_a} + \kappa)$ denote the probabilities of reporting the true value and any particular non-true value under KRR, respectively. For an honest node $v_h \notin \mathcal{M}$ with symmetric bilateral assessments, $\mathbb{E}[\phi_{v_h}] = 0$.

Proposition 2 establishes that the expected suspicion gap between malicious and honest nodes is $(p_a - q_a)(\kappa - 1)$, which grows with both ϵ_a (privacy budget) and κ (granularity of trust levels), providing a clear principle for hyperparameter selection. Finally, nodes whose suspicion score exceeds threshold γ are flagged and removed:

$$\hat{\mathcal{M}} = \{v \in \mathcal{V} : \phi_v > \gamma\}, \qquad \hat{\mathcal{V}} = \mathcal{V} \setminus \hat{\mathcal{M}}. \tag{16}$$

The pruned graph $\hat{\mathcal{G}} = (\hat{\mathcal{V}}, \hat{\mathcal{E}})$, where $\hat{\mathcal{E}} = \{(u, v) \in \hat{\mathcal{V}}^2 : c_{uv} > 0\}$, retains only nodes and edges with positive bilateral support and is passed to Stage ③.

C. Utility Restoration via Dual Denoising

After malicious node pruning, the retained graph $\hat{\mathcal{G}}$ still suffers from utility degradation induced by LDP perturbation. To recover signal quality, we adopt the *Node Feature Regularization* (NFR) and *High-Order Aggregator* (HOA) components from [15]: NFR reduces effective feature dimensionality via

Algorithm 3 VERITAS

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, privacy budgets ϵ_x, ϵ_a, trust levels κ, hyperparameters γ, λ, hops K, labeled nodes $\mathcal{V}_l$.
Output: Trained model $\mathcal{F}_\theta$, predictions on $\mathcal{V}_u$.

// Stage ①: Local Data Perturbation
for each user $v \in \mathcal{V}$ **do**
  $\tilde{\mathbf{x}}_v \leftarrow \mathcal{M}_x(\mathbf{x}_v, \epsilon_x)$ via Alg. 2. ▷ *feature perturbation*
  Construct $\breve{\mathbf{a}}_v \in \{0, 1, \ldots, \kappa\}^{|\mathcal{V}|}$ via Eq. (10).
  $\tilde{\mathbf{a}}_v \leftarrow \texttt{KRR}(\breve{\mathbf{a}}_v, \epsilon_a)$. ▷ *structure perturbation*
  Upload $(\tilde{\mathbf{x}}_v, \tilde{\mathbf{a}}_v)$ to server.
end for
// Stage ②: Attestation-Driven Malicious Node Pruning
for each candidate pair (u, v) **do**
  Extract signals s_{uv}^{exist}, s_{uv}^{cons}, s_{uv}^{asym} from $(\tilde{a}_{v,u}, \tilde{a}_{u,v})$.
  Compute edge confidence c_{uv} via Eq. (13).
end for
for each node $v \in \mathcal{V}$ **do**
  Compute suspicion score ϕ_v via Eq. (14).
end for
$\hat{\mathcal{M}} \leftarrow \{v \in \mathcal{V} : \phi_v > \gamma\}$; $\hat{\mathcal{V}} \leftarrow \mathcal{V} \setminus \hat{\mathcal{M}}$. ▷ *prune nodes*
Construct pruned graph $\hat{\mathcal{G}} = (\hat{\mathcal{V}}, \hat{\mathcal{E}})$.
// Stage ③: Utility Restoration via Dual Denoising
$\hat{\mathbf{X}} \leftarrow \texttt{NFR}(\{\tilde{\mathbf{x}}_v\}_{v \in \hat{\mathcal{V}}}, \epsilon_x)$. ▷ *node feature regularization*
$\hat{\mathbf{A}} \leftarrow \texttt{HOA}(\hat{\mathcal{G}}, K)$. ▷ *higher-order aggregator*
// Stage ④: Robust Private Graph Learning
Train $\mathcal{F}_\theta$ on $(\hat{\mathbf{X}}, \hat{\mathbf{A}})$ using labeled nodes $\mathcal{V}_l \cap \hat{\mathcal{V}}$.
Perform inference on $\mathcal{V}_u = \hat{\mathcal{V}} \setminus \mathcal{V}_l$.
return Trained model $\mathcal{F}_\theta$ and predictions on $\mathcal{V}_u$.

L_1-regularization to lower per-dimension data estimation error, while HOA expands the effective neighborhood through personalized multi-hop aggregation to reduce noise variance. Together, NFR and HOA produce a denoised feature matrix $\hat{\mathbf{X}}$ and refined adjacency structure $\hat{\mathbf{A}}$, which are subsequently fed into the robust private graph learning stage.

D. Robust Private Graph Learning

Upon receiving the denoised graph $(\hat{\mathbf{X}}, \hat{\mathbf{A}})$, the server trains a GNN $\mathcal{F}_\theta$ for downstream tasks such as node classification on the labeled node set $\mathcal{V}_l$, and subsequently performs inference on the unlabeled set $\mathcal{V}_u = \hat{\mathcal{V}} \setminus \mathcal{V}_l$. Since malicious nodes have been pruned and perturbation-induced noise has been mitigated in the preceding stages, the GNN operates on a substantially cleaner graph, yielding more reliable node representations and improved downstream utility. The complete VERITAS protocol is summarized in Algorithm 3.

E. Theoretical Analysis

1) Privacy Analysis: On the user side, perturbing node features and verification lists under ϵ_x-LDP and ϵ_a-link LDP, respectively, guarantees $(\epsilon_x + \epsilon_a)$-LDP by sequential composition (Theorem 2). On the server side, all subsequent operations are performed solely on the collected reports, and by post-processing invariance (Theorem 1), no additional privacy budget is consumed. Thus, VERITAS satisfies ϵ-LDP with $\epsilon = \epsilon_x + \epsilon_a$, providing an end-to-end formal privacy guarantee.

TABLE I
STATISTICS OF GRAPH DATASETS

Type	Dataset	#Nodes	#Edges	#Features	#Classes
Citation Network	Cora	2,708	5,278	1,433	7
	Citeseer	3,327	4,552	3,703	6
Social Network	LastFM	7,624	27,806	7,842	18
	Twitch	4,648	61,706	128	2

2) Complexity Analysis: The attestation-driven pruning stage (Stage ②) incurs a computational complexity of $\mathcal{O}(|\mathcal{V}| + |\mathcal{E}|)$ for bilateral signal extraction and suspicion score computation. The dual denoising stage (Stage ③) requires $\mathcal{O}(K|\mathcal{E}|d)$ and $\mathcal{O}(|\mathcal{V}|d)$ for HOA and NFR, respectively, where K denotes the number of propagation steps and d is the feature dimension. Consequently, both stages exhibit near-linear complexity with respect to the graph size, making VERITAS scalable to large-scale graph learning scenarios.

V. EXPERIMENTS

In this section, we empirically evaluate the effectiveness of VERITAS. Sect. V-A outlines the experimental settings, while Sect. V-B reports the results and provides detailed discussions.

A. Experimental Settings

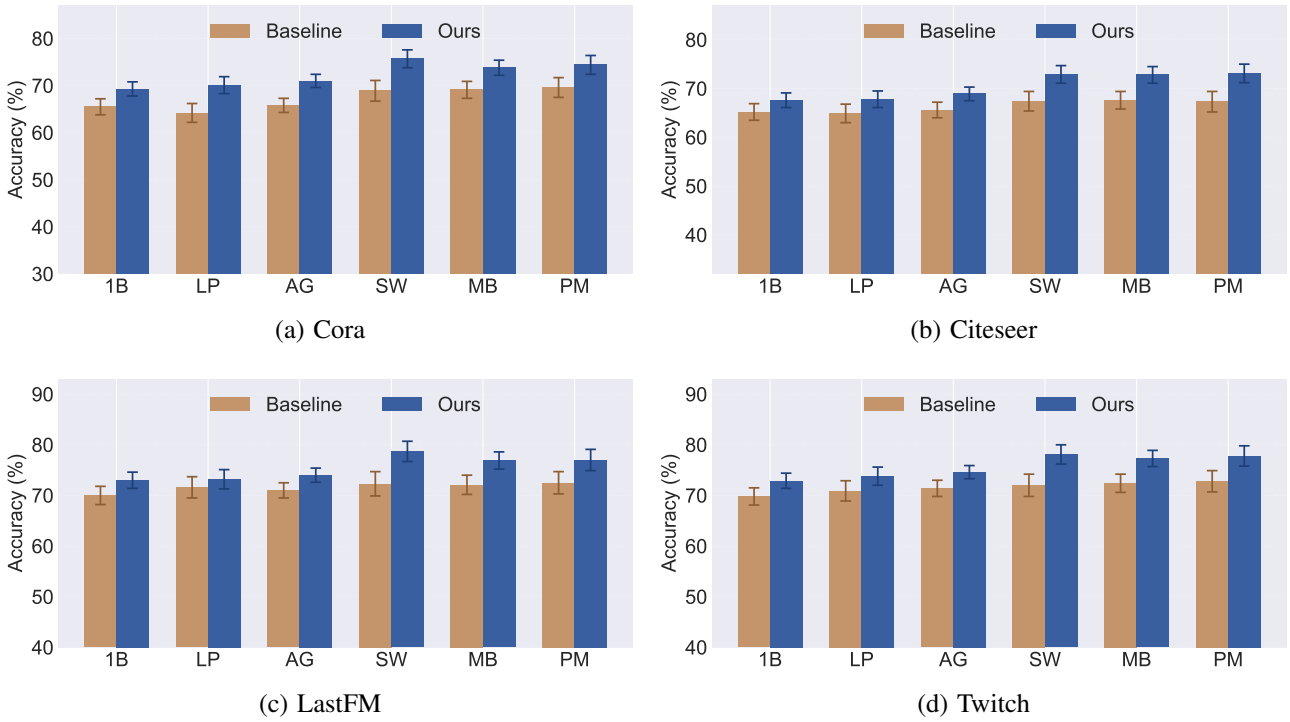
1) Datasets: We conduct experiments on four benchmark graph datasets spanning two categories: two citation networks, *Cora* [42] and *Citeseer* [42], where nodes represent papers and edges denote citation relationships; and two social networks, *LastFM* [43] and *Twitch* [43], where nodes represent users and edges encode social links. These datasets have been broadly adopted for evaluating graph learning methods under privacy constraints [14]–[16]. Table I summarizes their statistics.

2) GNN Models: We consider three representative GNN backbones: GCN [2], GraphSAGE [1], and GAT [44]. Unless otherwise specified, each model is implemented with two graph convolutional layers with hidden dimension 64. We implement the GNN models in PyTorch using PyTorch-Geometric (PyG)[2]. The GCN serves as the default backbone.

3) LDP Mechanisms: We consider six LDP mechanisms for node feature protection, covering both classical and recent approaches. The classical baselines include the 1-bit mechanism (1B) [28], the Laplace mechanism (LP) [36], and the Analytic Gaussian mechanism (AG) [37]. We further include three advanced mechanisms that have demonstrated strong utility-privacy trade-offs in graph settings: the piecewise mechanism (PM) [39], the multi-bit mechanism (MB) [13], and the square wave mechanism (SW) [38]. PM is used as the default mechanism. For graph structure protection, the verification list is perturbed via KRR as described in Sect. IV-A.

4) Parameter Settings: The feature privacy budget ϵ_x is varied over $\{0.01, 0.1, 1.0, 2.0, 3.0\}$ and the structural privacy budget ϵ_a over $\{1.0, 3.0, 5.0, 7.0, 9.0\}$, with defaults $\epsilon_x = 0.1$ and $\epsilon_a = 5.0$. All datasets are split into 50% training, 25%

[2]https://www.pyg.org/



(a) Cora (b) Citeseer

(c) LastFM (d) Twitch

Fig. 3. Performance comparison between our proposed VERITAS (Ours) and attack baseline on the node classification task.

TABLE II
COMPARISON OF CLASSIFICATION ACCURACY BETWEEN OUR DEFENSE AND ATTACK BASELINE ACROSS DIFFERENT GNN MODELS.

GNN	Method	Cora	Citeseer	LastFM	Twitch
GraphSAGE	Baseline	69.2	67.6	72.4	72.7
	Ours	**75.7**	**72.9**	**76.3**	**76.9**
GAT	Baseline	67.4	65.1	69.6	70.1
	Ours	**71.8**	**70.3**	**75.2**	**75.4**

validation, and 25% test sets. The granularity is $\kappa = 5$. The injection rate $r^\star$ is varied over $\{0.001, 0.005, 0.01, 0.02, 0.04\}$ with default $r^\star = 0.01$. The threshold γ and other parameters are tuned on the validation set. Utility restoration follows [15].

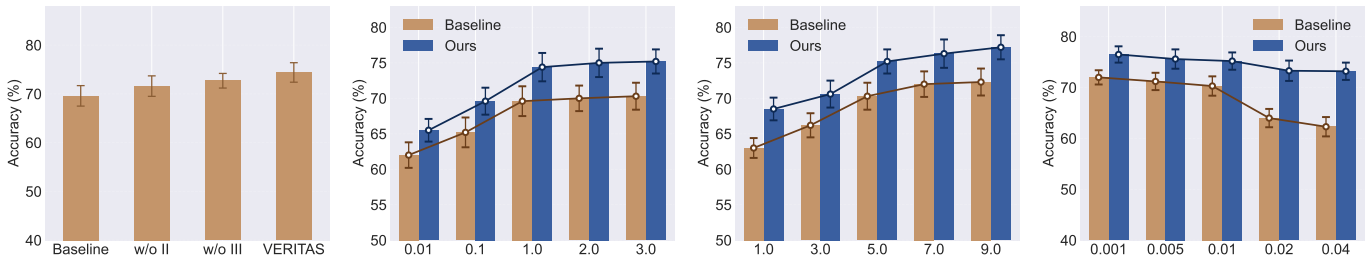
5) Evaluation Metrics: We evaluate task performance on the node classification task [2], measured by test accuracy (%). All results are averaged over 10 independent runs with 95% confidence intervals. Experiments are run on a server equipped with Ubuntu 20.04 LTS, two Intel® Xeon® Gold 6348 CPUs, 100GB RAM, and an NVIDIA® A800 80GB GPU.

B. Experimental Results

1) Effectiveness: We evaluate the effectiveness of VERITAS against data poisoning attacks on four benchmark datasets under the default privacy budget setting, using the poisoned model without defense as the baseline. As shown in Fig. 3, VERITAS consistently improves node classification accuracy across all datasets and LDP mechanisms, demonstrating strong resilience to poisoning attacks under diverse perturbation schemes. Table II further verifies the robustness of VERITAS across different GNN architectures. Under both GraphSAGE and GAT backbones, our framework substantially restores the utility degraded by poisoning, indicating that its effectiveness is largely independent of the underlying GNN design. This is mainly because VERITAS operates at the data and participant filtering level rather than modifying model-specific architectures, allowing it to generalize across different GNN encoders. Overall, these results demonstrate the strong generalizability of VERITAS across both privacy mechanisms and GNN models.

2) Ablation Studies: To quantify the contribution of each core defense stage, we compare four configurations: **Baseline** (poisoned without any defense), **w/o II** (full VERITAS without attestation-driven malicious node pruning), **w/o III** (full VERITAS without dual denoising), and the complete **VERITAS**. As shown in Fig. 4(a), removing either stage leads to a noticeable performance drop relative to the full model: removing Stage II causes the most significant degradation, confirming that malicious node pruning is the primary defense mechanism; removing Stage III also reduces accuracy, highlighting the importance of utility restoration. The full VERITAS achieves the best performance, demonstrating that the two stages are complementary and both indispensable.

3) Parameter Analysis: We analyze the sensitivity of VERITAS to three key parameters: the feature privacy budget ϵ_x, the structural privacy budget ϵ_a, and the injection rate $r^\star$. As shown in Fig. 4(b) and (c), the node classification accuracy consistently improves as either ϵ_x or ϵ_a increases. This is because larger privacy budgets introduce less perturbation noise, preserving more informative feature and structural signals for malicious node detection. Fig. 4(d) evaluates the effect of the $r^\star$. We observe that VERITAS maintains stable performance



(a) Ablation (b) Parameter ϵ_x (c) Parameter ϵ_a (d) Parameter $r^\star$

Fig. 4. Analysis of VERITAS under data poisoning attacks. (a) Ablation study under different defense configurations, evaluating the contribution of each core component. (b)–(d) Parameter sensitivity with respect to the feature privacy budget ϵ_x, structural privacy budget ϵ_a, and injection rate $r^\star$, respectively.

when the proportion of malicious nodes is relatively low. Although the accuracy gradually decreases as $r^\star$ increases, the degradation remains moderate, indicating that VERITAS can effectively withstand increasingly strong poisoning attacks.

VI. Related Work

In this section, we review the existing literature related to local differential privacy (▷ VI-A), graph neural networks (▷ VI-B), and locally private graph learning (▷ VI-C).

A. Local Differential Privacy

LDP [24] has emerged as a *de facto* standard for privacy-preserving data collection and analysis in decentralized environments, as it eliminates the need for a trusted data curator. In the LDP setting, each user perturbs their sensitive data before transmission, ensuring rigorous privacy guarantees even against an untrusted server. Owing to its strong privacy protection and practical deployability, LDP has attracted extensive attention from both academia and industry [45], [46]. Representative mechanisms include randomized response [40], [41] for categorical data, as well as the Laplace [36], Gaussian [37], piecewise [39], multi-bit [13], and square wave [38] mechanisms for numerical data. Building upon these primitives, numerous studies have investigated private frequency estimation, distribution learning, and machine learning under LDP constraints. The practicality of LDP has also led to its large-scale deployment in real-world systems by major technology companies, including Google [26], Apple [27], and Microsoft [28]. More recently, the scope of LDP has been extended from tabular data to graph-structured data [14], [21], [35], laying the foundation for locally private graph learning.

B. Graph Neural Networks

GNNs [2], [3] have emerged as the dominant framework for learning over graph-structured data, achieving remarkable success in a wide range of graph mining tasks [2], [4], [5]. By iteratively aggregating information from neighboring nodes, GNNs effectively capture both topological dependencies and node attributes, yielding expressive graph representations [1], [2], [44]. Consequently, GNNs have been widely adopted in numerous real-world applications [6]–[8]. Despite their effectiveness, GNNs are frequently deployed on graphs containing sensitive user attributes and relational information, raising significant privacy concerns. Recent studies have shown that adversaries can exploit trained GNN models to infer private information, giving rise to various privacy attacks, including attribute inference [9], membership inference [12], and link inference [10]. These risks have motivated a growing body of research on privacy-preserving graph learning, among which locally private graph learning has attracted particular attention.

C. Locally Private Graph Learning

Locally private graph learning extends the LDP paradigm to graph-structured data, enabling collaborative graph learning without revealing raw node features or graph topology to the server [13]–[23]. By leveraging the inherent noise tolerance of GNN message passing [2], [13], these methods can effectively learn useful node representations from perturbed graph data, maintaining strong utility under rigorous privacy constraints. Despite these advantages, the decentralized nature of locally private graph learning introduces unique security vulnerabilities. Recent studies [16] have shown that malicious participants can exploit both feature and structural perturbation channels to inject adversarial signals, which are subsequently propagated and amplified through the neighborhood aggregation process of GNNs, leading to significant utility degradation. Defending against such attacks is particularly challenging in the local privacy setting. Unlike centralized graph learning, where anomaly detection methods can leverage global graph statistics and feature correlations [30], [32], [33], LDP mechanisms deliberately obfuscate both structural and attribute information, rendering many existing defenses ineffective. These challenges highlight a critical gap in the security of locally differentially private graph learning and underscore the urgent need for defense mechanisms specifically tailored to the LDP setting.

VII. Conclusion

In this paper, we propose VERITAS, a poisoning-resilient locally private graph learning protocol built upon a *trust-but-verify* paradigm. By leveraging the asymmetry of bilateral attestation signals, VERITAS effectively identifies and prunes malicious participants while preserving rigorous LDP guarantees. Extensive experiments on four datasets under multiple LDP mechanisms and GNN architectures demonstrate that

VERITAS consistently defends against poisoning attacks and significantly improves task utility. These results highlight the importance of integrating security considerations into locally private graph learning and establish a promising direction for building robust privacy-preserving graph learning systems.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2024YFF0907401), the National Natural Science Foundation of China (No. 62072052), and the BUPT Excellent Ph.D. Students Foundation (No. CX20260030).

REFERENCES

[1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, vol. 30, pp. 1024–1034, 2017.

[2] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017, pp. 1–15.

[3] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, vol. 32, no. 1, pp. 4–24, 2020.

[4] W. Liu, K. Yue, Z. Zhang, and L. Duan, "Hierarchical structure enhanced graph representation for link prediction," in *ICDM*, 2025, pp. 507–516.

[5] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *Journal of Machine Learning Research (JMLR)*, vol. 24, no. 127, pp. 1–21, 2023.

[6] K. Sharma, Y.-C. Lee, S. Nambi, A. Salian, S. Shah, S.-W. Kim, and S. Kumar, "A survey of graph neural networks for social recommender systems," *ACM Computing Surveys (CSUR)*, vol. 56, no. 10, pp. 1–34, 2024.

[7] L. Jain, R. Katarya, and S. Sachdeva, "Opinion leaders for information diffusion using graph neural network in online social networks," *ACM Transactions on the Web (TWEB)*, vol. 17, no. 2, pp. 1–37, 2023.

[8] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, "Pick and choose: a gnn-based imbalanced learning approach for fraud detection," in *WWW*, 2021, pp. 3168–3177.

[9] L. Meng, Y. Bai, Y. Chen, Y. Hu, W. Xu, and H. Weng, "Devil in disguise: Breaching graph neural networks privacy through infiltration," in *CCS*, 2023, pp. 1153–1167.

[10] F. Wu, Y. Long, C. Zhang, and B. Li, "Linkteller: Recovering private edges from graph neural networks via influence analysis," in *S&P*, 2022, pp. 2005–2024.

[11] X. Wang and W. H. Wang, "Group property inference attacks against graph neural networks," in *CCS*, 2022, pp. 2871–2884.

[12] Z. Zhang, M. Chen, M. Backes, Y. Shen, and Y. Zhang, "Inference attacks against graph neural networks," in *USENIX Security*, 2022, pp. 4543–4560.

[13] S. Sajadmanesh and D. Gatica-Perez, "Locally private graph neural networks," in *CCS*, 2021, pp. 2130–2145.

[14] W. Lin, B. Li, and C. Wang, "Towards private learning on decentralized graphs with local differential privacy," *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 17, pp. 2936–2946, 2022.

[15] L. He, C. Li, P. Tang, and S. Su, "Going deeper into locally differentially private graph neural networks," in *ICML*, 2025, pp. 22 548–22 565.

[16] L. He, C. Li, P. Tang, L. Sun, S. Su, and P. S. Yu, "Devil's hand: Data poisoning attacks to locally private graph learning protocols," in *KDD*, 2026, pp. 395–406.

[17] S. Hidano and T. Murakami, "Degree-preserving randomized response for graph neural networks under local differential privacy," *Transactions on Data Privacy (TDP)*, vol. 17, no. 2, pp. 89–121, 2024.

[18] Z. Li, R.-H. Li, M. Liao, F. Jin, and G. Wang, "Privacy-preserving graph embedding based on local differential privacy," in *CIKM*, 2024, pp. 1316–1325.

[19] X. Pei, X. Deng, S. Tian, J. Liu, and K. Xue, "Privacy-enhanced graph neural network for decentralized local graphs," *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 19, pp. 1614–1629, 2023.

[20] L. He, P. Tang, L. Sun, and S. Su, "The devil within, the cure without: Securing locally private graph learning under poisoning," in *WWW*, 2026, pp. 4493–4504.

[21] X. Zhu, V. Y. Tan, and X. Xiao, "Blink: Link local differential privacy in graph neural networks via bayesian estimation," in *CCS*, 2023, pp. 2651–2664.

[22] L. He, L. Sun, J. Wu, S. Su, and P. S. Yu, "Differentially private graph learning: A survey," *TechRxiv*, 2026.

[23] L. He, P. Tang, C. Li, J. Fu, L. Zhang, L. Sun, P. S. Yu, and S. Su, "Toward personalized differentially private learning for decentralized local graphs," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 38, pp. 6255–6269, 2026.

[24] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SIAM Journal on Computing (SICOMP)*, vol. 40, no. 3, pp. 793–826, 2011.

[25] C. Dwork, "Differential privacy: A survey of results," in *TAMC*, 2008, pp. 1–19.

[26] Ú. Erlingsson, V. Pihur, and A. Korolova, "Rappor: Randomized aggregatable privacy-preserving ordinal response," in *CCS*, 2014, pp. 1054–1067.

[27] A. G. Thakurta, A. H. Vyrros, U. S. Vaishampayan, G. Kapoor, J. Freudiger, V. R. Sridhar, and D. Davidson, "Learning new words, us patent 9,594,741," 2017.

[28] B. Ding, J. Kulkarni, and S. Yekhanin, "Collecting telemetry data privately," *NeurIPS*, vol. 30, pp. 3571–3580, 2017.

[29] J. R. Douceur, "The sybil attack," in *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.

[30] L. Sun, Y. Dou, C. Yang, K. Zhang, J. Wang, P. S. Yu, L. He, and B. Li, "Adversarial attack and defense on graph data: A survey," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 35, no. 8, pp. 7693–7711, 2022.

[31] J. Ma, S. Ding, and Q. Mei, "Towards more practical adversarial attacks on graph neural networks," *NeurIPS*, vol. 33, pp. 4756–4766, 2020.

[32] X. Zhang and M. Zitnik, "Gnnguard: Defending graph neural networks against adversarial attacks," *NeurIPS*, vol. 33, pp. 9263–9275, 2020.

[33] X. Tang, Y. Li, Y. Sun, H. Yao, P. Mitra, and S. Wang, "Transferring robustness for graph neural network against poisoning attacks," in *WSDM*, 2020, pp. 600–608.

[34] L. He, P. Tang, Y. Zhang, P. Zhou, and S. Su, "Mitigating privacy risks in retrieval-augmented generation via locally private entity perturbation," *Information Processing & Management*, vol. 62, no. 4, p. 104150, 2025.

[35] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *CCS*, 2017, pp. 425–438.

[36] N. Phan, X. Wu, H. Hu, and D. Dou, "Adaptive laplace mechanism: Differential privacy preservation in deep learning," in *ICDM*, 2017, pp. 385–394.

[37] B. Balle and Y.-X. Wang, "Improving the gaussian mechanism for differential privacy: Analytical calibration and optimal denoising," in *ICML*, 2018, pp. 394–403.

[38] Z. Li, T. Wang, M. Lopuhaä-Zwakenberg, N. Li, and B. Škoric, "Estimating numerical distributions under local differential privacy," in *SIGMOD*, 2020, pp. 621–635.

[39] N. Wang, X. Xiao, Y. Yang, J. Zhao, S. C. Hui, H. Shin, J. Shin, and G. Yu, "Collecting and analyzing multidimensional data with local differential privacy," in *ICDE*, 2019, pp. 638–649.

[40] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," *Journal of the American Statistical Association (JASA)*, vol. 60, no. 309, pp. 63–69, 1965.

[41] P. Kairouz, K. Bonawitz, and D. Ramage, "Discrete distribution estimation under local privacy," in *ICML*, 2016, pp. 2436–2444.

[42] Z. Yang, W. Cohen, and R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," in *ICML*, 2016, pp. 40–48.

[43] B. Rozemberczki and R. Sarkar, "Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models," in *CIKM*, 2020, pp. 1325–1334.

[44] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018, pp. 1–15.

[45] X. Xiong, S. Liu, D. Li, Z. Cai, and X. Niu, "A comprehensive survey on local differential privacy," *Security and Communication Networks*, vol. 2020, no. 1, p. 8829523, 2020.

[46] G. Cormode, S. Jha, T. Kulkarni, N. Li, D. Srivastava, and T. Wang, "Privacy at scale: Local differential privacy in practice," in *SIGMOD*, 2018, pp. 1655–1658.